

Guía: de macro a video

Simulación de termalización de neutrones con fuente AmBe

Geant4 + Python + ffmpeg

Proyecto:	Neutron_Thermalization
Branch:	<code>ambe_source</code>
Simulador:	Geant4 11.04-beta-01 (OGLSX + gl2ps 1.4.2)
Entorno Python:	<code>mi_entorno</code> (virtualenv)
Fecha:	Mayo 2026

Resumen del flujo:

1. Editar `vis_video.mac` (geometría, fuente, colores).
2. Correr Geant4 → genera 120 archivos EPS por evento.
3. Convertir EPS → PNG con Ghostscript.
4. Acumular frames con Python (`accumulate_frames.py`).
5. Compilar video MP4 con `ffmpeg`.

Índice

1. Estructura de archivos relevantes	2
2. El macro principal: vis_video.mac	2
2.1. Parámetros que querrás cambiar	3
2.2. Sub-macro de frame: vis_video_frame.mac	3
3. Paso 1 — Generar el frame de referencia del cubo	3
4. Paso 2 — Convertir EPS a PNG con Ghostscript	4
5. Paso 3 — Acumular frames con Python	5
5.1. Cómo funciona la acumulación	5
6. Paso 4 — Compilar el video con ffmpeg	5
7. Paso 5 — Limpieza de archivos intermedios	6
8. Flujo completo en un solo bloque	6
9. Solución de problemas comunes	7
10. Referencia rápida de colores de partículas	7

1 Estructura de archivos relevantes

Archivo	Función
macros/vis_video.mac	Macro principal: geometría, fuente, escena, loop de eventos
macros/vis_video_frame.mac	Sub-macro: un evento + flush + export EPS
python/accumulate_frames.py	Acumula PNGs frame a frame con overlay del cubo
build/	Directorio de compilación; aquí se genera todo

Los comandos de esta guía se ejecutan **siempre desde build/**, salvo que se indique lo contrario.

2 El macro principal: vis_video.mac

A continuación se muestra el macro completo con comentarios explicativos.

Listing 1: macros/vis_video.mac — macro completo

```
# vis_video.mac      120 eventos, acumulaci n part cula a part cula
/control/macroPath ../macros
/control/verbose 0
/run/verbose 0
/event/verbose 0
/tracking/verbose 0

# Geometr a
/detector/setParaffinX 10 cm
/detector/setParaffinY 10 cm
/detector/setParaffinZ 10 cm

# Fuente AmBe
/ambe/distance 10 cm
/ambe/activity 2.98

/run/initialize

# Visualizaci n
/vis/open OGLSX 1280x720
/vis/viewer/set/viewpointThetaPhi 30 20
/vis/viewer/set/background 0 0 0 1
/vis/geometry/set/colour Block 0 0.47 0.43 0.35 1.0
/vis/geometry/set/colour Detector 0 1.0 0.55 0.0 1.0
/vis/viewer/set/style wireframe
/vis/geometry/set/forceSolid Detector 0
/vis/scene/create
/vis/scene/add/volume
/vis/drawVolume
/vis/scene/add/trajectories smooth
/vis/scene/add/hits
/vis/scene/endOfEventAction accumulate
/vis/scene/endOfRunAction accumulate

# Colores por tipo de part cula
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set neutron white
/vis/modeling/trajectories/drawByParticleID-0/set gamma yellow
```

```

/vis/modeling/trajectories/drawByParticleID-0/set e- red

# Etiqueta
/vis/set/textColour green
/vis/set/textLayout right
/vis/scene/add/text2D 0.9 -0.9 18 ! ! AmBe Neutron Thermalization
/vis/set/textLayout
/vis/set/textColour
/vis/scene/add/axes -10 -10 -10 10 cm

/vis/enable
/vis/viewer/refresh
/vis/ogl/set/exportFormat eps

# 120 eventos, 1 por frame      60 fps      2 segundos
/control/loop vis_video_frame.mac i 1 120 1

```

2.1 Parámetros que querrás cambiar

Parámetro	Línea en el macro	Efecto
Tamaño de la caja de parafina	<code>/detector/setParaffinX/Y/Z</code>	Dimensiones del moderador (half-e)
Distancia fuente–detector	<code>/ambe/distance</code>	Posición de la fuente AmBe a lo la
Actividad	<code>/ambe/activity</code>	Factor de escala del espectro (neut)
Ángulo de vista	<code>/vis/viewer/set/viewpointThetaPhi</code>	θ (elevación) y ϕ (azimut) en grad
Color caja de parafina	<code>/vis/geometry/set/colour Block</code>	R G B A, todos en [0,1]
Color detector	<code>/vis/geometry/set/colour Detector</code>	R G B A, todos en [0,1]
Número de eventos	<code>/control/loop ... 1 120 1</code>	Cambiar 120 al número de frames

Importante — colores en Geant4: el comando `/vis/geometry/set/colour` espera valores flotantes en [0,1], *no* enteros RGB de 0–255. Usar valores como 79 74 64 los clampea a 1.0 (blanco puro). Usa siempre valores como 0.47 0.43 0.35.

2.2 Sub-macro de frame: vis_video_frame.mac

Este archivo es invocado por el `/control/loop` del macro principal. La variable `{i}` se sustituye automáticamente con el número de iteración.

Listing 2: macros/vis_video_frame.mac

```

/run/beamOn 1
/vis/viewer/flush
/vis/ogl/export frame_{i}.eps

```

Cada iteración: corre 1 evento, vuelca la escena y exporta un EPS nombrado `frame_{i}.eps` en el directorio de trabajo (`build/`).

3 Paso 1 — Generar el frame de referencia del cubo

Antes de lanzar los 120 eventos, se genera **un único frame** con la caja de parafina sólida y sin trayectorias. Este PNG se usará después como overlay semitransparente para que los bordes del cubo sean visibles.

Listing 3: Generar box_frame.png desde build/

```
# Dentro de build/, con el ejecutable ya compilado:
./neutron_therm ../macros/vis_video.mac
# Geant4 abre OGLSX, configura la escena y ejecuta el loop.
# Al terminar hay 120 archivos frame_{i}.eps en build/
```

Si sólo quieres regenerar `box_frame.png` (por ejemplo, para cambiar el color de la caja sin re-correr la simulación completa), crea un macro mínimo:

```
# box_only.mac
/detector/setParaffinX 10 cm
/detector/setParaffinY 10 cm
/detector/setParaffinZ 10 cm
/ambe/distance 10 cm
/ambe/activity 2.98
/run/initialize
/vis/open OGLSX 1280x720
/vis/viewer/set/viewpointThetaPhi 30 20
/vis/viewer/set/background 0 0 0 1
/vis/geometry/set/colour Block 0 0.65 0.60 0.48 1.0
/vis/geometry/set/colour Detector 0 1.0 0.55 0.0 1.0
/vis/viewer/set/style surface
/vis/scene/create
/vis/scene/add/volume
/vis/drawVolume
/vis/enable
/vis/viewer/refresh
/vis/ogl/set/exportFormat eps
/vis/ogl/export box_frame_0000.eps
```

y convierte con Ghostscript (ver Paso 2).

4 Paso 2 — Convertir EPS a PNG con Ghostscript

Geant4 exporta en formato EPS (Encapsulated PostScript). Ghostscript convierte cada EPS a un PNG de 1280×720.

Listing 4: Conversión por lotes — desde build/

```
# Frame de referencia del cubo
gs -dNOPAUSE -dBATCH -sDEVICE=png16m -r72 -dEPSCrop \
  -sOutputFile=box_frame.png box_frame_0000.eps

# Los 120 frames de trayectorias
for i in $(seq -f "%04g" 1 120); do
  gs -dNOPAUSE -dBATCH -sDEVICE=png16m -r72 -dEPSCrop \
    -sOutputFile=frame_${i}.png frame_${i}.eps
done
```

Flag gs	Significado
-dNOPAUSE -dBATCH	Modo no interactivo
-sDEVICE=png16m	Salida PNG de 24 bits
-r72	72 dpi (ajusta el tamaño de píxeles a 1280×720)
-dEPSCrop	Recorta al bounding box del EPS

Si quieres mayor resolución, sube **-r144** (doble) o **-r216** (triple), y ajusta correspondientemente el tamaño de ventana OGLSX en el macro.

5 Paso 3 — Acumular frames con Python

El script `accumulate_frames.py` genera los frames acumulados: el frame N muestra las trayectorias de los eventos 1 a N superpuestas, con el cubo de referencia como overlay tenue.

Listing 5: Acumulación — desde build/

```
source ~/mi_entorno/bin/activate

python3 ../python/accumulate_frames.py \
  --n 120 \
  --box box_frame.png \
  --box-alpha 0.25
```

Argumento	Descripción
<code>-n 120</code>	Número total de frames (debe coincidir con el loop del macro)
<code>-box box_frame.png</code>	PNG del cubo sólido de referencia
<code>-box-alpha 0.25</code>	Opacidad del overlay del cubo (0 = invisible, 1 = opaco)
<code>-indir .</code>	Directorio donde están los <code>frame_NNNN.png</code> (default: .)
<code>-outdir .</code>	Directorio de salida para los <code>accum_NNNN.png</code> (default: .)

Esto genera `accum_0001.png ... accum_0120.png`.

5.1 Cómo funciona la acumulación

1. Para cada frame N : se calcula `np.maximum(acumulado, frame_N)`, manteniendo el pixel más brillante de todos los eventos anteriores.
2. Se aplica el overlay: `np.maximum(acumulado, cubo × alpha)`. Como las trayectorias son blancas/amarillas (valor 255) y el overlay es tenue (~ 40), el `maximum` siempre elige la trayectoria donde hay una; el cubo sólo se ve donde *no* hay trayectorias.
3. El resultado se guarda como `accum_NNNN.png`.

Para cambiar sólo la opacidad del cubo sin re-correr la simulación, basta con ajustar `-box-alpha` y re-ejecutar este paso. Los `frame_NNNN.png` ya están en disco.

6 Paso 4 — Compilar el video con ffmpeg

Listing 6: Compilar video MP4 — desde build/

```
ffmpeg -y \
  -framerate 60 \
  -start_number 1 \
  -i accum_%04d.png \
```

```
-c:v libx264 \
-pix_fmt yuv420p \
-crf 18 \
video_simulacion.mp4
```

Flag ffmpeg	Descripción
-framerate 60	60 fps $\Rightarrow$ 120 eventos = 2 segundos de video
-start_number 1	El primer archivo es <code>accum_0001.png</code>
-c:v libx264	Codec H.264 (compatible con casi todo)
-pix_fmt yuv420p	Formato de color compatible con reproductores
-crf 18	Calidad alta (0 = lossless, 51 = mínima)

Para cambiar la duración del video, ajusta `-framerate`: a 30 fps serían 4 segundos; a 15 fps, 8 segundos.

7 Paso 5 — Limpieza de archivos intermedios

Los archivos EPS y los PNGs individuales pueden ocupar varios cientos de MB. Una vez compilado el video, se pueden eliminar:

Listing 7: Limpieza — desde build/

```
rm -f frame_*.eps frame_*.png accum_*.png box_frame*.eps

# Verificar que el video qued bien
ls -lh video_simulacion.mp4
```

8 Flujo completo en un solo bloque

El siguiente bloque ejecuta todo el pipeline de una vez, asumiendo que ya se editó `vis_video.mac` y que el ejecutable está compilado en `build/`.

Listing 8: Pipeline completo — desde build/

```
# 1. Simulaci n Geant4 (genera frame_1.eps ... frame_120.eps y
    box_frame_0000.eps)
./neutron_therm ../macros/vis_video.mac

# 2. Conversi n EPS -> PNG
gs -dNOPAUSE -dBATCH -sDEVICE=png16m -r72 -dEPSCrop \
-sOutputFile=box_frame.png box_frame_0000.eps

for i in $(seq -f "%04g" 1 120); do
    gs -dNOPAUSE -dBATCH -sDEVICE=png16m -r72 -dEPSCrop \
-sOutputFile=frame_${i}.png frame_${i}.eps
done

# 3. Acumulaci n con overlay del cubo
source ~/mi_entorno/bin/activate
python3 ../python/accumulate_frames.py \
--n 120 --box box_frame.png --box-alpha 0.25

# 4. Compilar video
ffmpeg -y -framerate 60 -start_number 1 -i accum_%04d.png \
```

```
-c:v libx264 -pix_fmt yuv420p -crf 18 video_simulacion.mp4

# 5. Limpiar intermedios
rm -f frame_*.eps frame_*.png accum_*.png box_frame*.eps

echo "Listo: video_simulacion.mp4"
```

9 Solución de problemas comunes

Síntoma	Causa y solución
Parafina siempre blanca aunque cambies el color	Los valores RGB no están en $[0, 1]$. Geant4 clampea integers al máximo. Usa 0.47 0.43 0.35, no 79 74 64.
Los bordes del cubo salen azules en modo wireframe	Limitación de <code>gl2ps 1.4.2</code> : ignora el color asignado a wireframes y los renderiza en azul. La solución adoptada es el overlay en Python.
Trayectorias tapadas por la cara frontal de la caja	Geant4 usa el algoritmo del pintor; la cara frontal se dibuja encima. Solución: modo <code>wireframe</code> en el macro + overlay en Python.
Los PNG salen en blanco o muy pequeños	Ajusta <code>-r72</code> en Ghostscript. Prueba <code>-r96</code> si la imagen queda pequeña.
El video dura muy poco o va muy rápido	Baja el framerate: <code>-framerate 30</code> (4 s) o <code>-framerate 15</code> (8 s).
<code>./neutron_therm:</code> <code>command not found</code>	Falta compilar. Desde <code>build/</code> : <code>cmake .. && make -j4</code> .

10 Referencia rápida de colores de partículas

Los colores de las trayectorias se definen en `vis_video.mac`:

```
/vis/modeling/trajectories/create/drawByParticleID
/vis/modeling/trajectories/drawByParticleID-0/set neutron white
/vis/modeling/trajectories/drawByParticleID-0/set gamma yellow
/vis/modeling/trajectories/drawByParticleID-0/set e- red
```

Para agregar otras partículas (p.ej., protones de retroceso):

```
/vis/modeling/trajectories/drawByParticleID-0/set proton cyan
```

Los nombres de colores aceptados por Geant4 incluyen: `white`, `black`, `red`, `green`, `blue`, `yellow`, `cyan`, `magenta`, `grey`, `orange`.